PRESENTACIÓN DEL PRODUCTO · VERSIÓN 1.1.1

QA Scout
Explora con una misión. Vuelve con evidencia.

Extensión de Chrome, gratuita y de código abierto, para pruebas exploratorias basadas en sesiones. Registra lo que pasa mientras exploras y entrega al equipo el informe y el bug listos.

0
solicitudes de red: los datos se quedan en tu ordenador

4
idiomas: pt-BR, pt-PT, inglés y español

176
pruebas automatizadas en el propio proyecto

Thamires Azevedo · QA Senior · github.com/azevedothamirest/qa-scout

EL CONCEPTO

¿Qué es la prueba exploratoria?

Es aprender sobre el sistema, diseñar las pruebas y ejecutarlas **al mismo tiempo**. En lugar de seguir un guion cerrado, quien prueba investiga con una misión, observa el comportamiento real y decide el siguiente paso según lo que acaba de ver.

Guion (script)

Comprueba lo que ya se sabe que debería pasar. Bueno para regresión y automatización.

Exploratoria

Descubre lo que nadie pensó en escribir: combinaciones, casos límite, flujos reales. Ahí aparecen los bugs más caros.

Sesiones con tiempo acotado (SBTM)

El **Session-Based Test Management**, de Jon y James Bach, da estructura sin convertirse en guion. Cada sesión tiene tres partes:

1. **Charter:** la misión, en el modelo de Elisabeth Hendrickson.
2. **Sesión:** un bloque de tiempo sin interrupciones (30 a 90 min), anotando bugs, dudas, ideas, riesgos y observaciones.
3. **Debriefing:** revisión de las notas, reparto del tiempo en Test, Bug y Setup (TBS) y lo que queda para la próxima sesión.

Explorar **el checkout con pago instantáneo** con **QR caducado, botón atrás y cupón** para descubrir **pedidos pagados que quedan pendientes**.

Reparto del tiempo (TBS)



Prueba 60% · Bug 25% · Setup 15%. Mucho tiempo en setup o bugs indica un entorno inestable o un producto frágil: una métrica que todo el equipo entiende.

EL PROBLEMA

La prueba exploratoria suele ser trabajo invisible

Encuentra los mejores bugs, pero deja poco rastro: ¿qué se exploró, durante cuánto tiempo, con qué evidencia? Y reproducir el bug después se vuelve arqueología.

LA SOLUCIÓN

QA Scout anota mientras exploras

📸 Captura automática

Clics, campos rellenados y páginas abiertas se vuelven notas con captura, en el panel lateral de Chrome. **El valor escrito nunca se lee.**

🧯 Errores de consola y red

Errores de JavaScript, `console.error`, respuestas 4××/5×× y fallos de conexión se vuelven notas sugeridas como bug o riesgo.

🪜 Pasos para reproducir

Un clic arma la lista con las acciones previas al bug.

🐞 Bug listo para Jira

"Copiar bug" genera título, entorno, navegador, página, pasos, obtenido vs. esperado y los errores del mismo minuto.

🃏 Baraja de heurísticas

18 cartas (tours de Whittaker, datos, interacción, contexto) para cuando te bloquees.

📄 Informe de la sesión

Debriefing con TBS e informe en Markdown o PDF para pegar en Jira, Notion o Slack.

Tú lo revisas todo. Las notas automáticas llegan "por revisar": confirma, edita o elimina. Solo las revisadas entran en el informe.

PARA TODO EL EQUIPO

Cómo lo aplica cada área

La prueba exploratoria no es exclusiva de QA. Con una misión corta y 20 minutos, cualquier persona del equipo encuentra lo que la prueba con guion no detecta.

QA

Sesiones con misión y evidencia

- Planifica el sprint en charters por riesgo: ¿qué puede salir más caro?
- Usa el TBS en el debriefing para mostrar adónde fue el tiempo.
- Abre bugs con pasos y errores de consola ya adjuntos, sin "no lo puedo reproducir".
- Convierte lo que la sesión descubrió en casos de regresión automatizada.

Dev

Explorar antes de abrir el PR

- Una sesión de 15 min en tu propia feature: "explorar el formulario nuevo con datos inválidos para descubrir validaciones que faltan".
- Los bugs llegan con pasos, entorno, URL y el error de JavaScript del mismo minuto: menos idas y vueltas.
- En la revisión del PR, explora la rama en staging y pega el informe en el PR.

Producto

Validar la experiencia real

- Recorre el camino como cliente antes del lanzamiento: "explorar el onboarding con un usuario nuevo para descubrir dónde se atasca".
- Las notas de **duda** e **idea** se vuelven preguntas de refinamiento y backlog.
- El informe muestra cobertura y riesgos en lenguaje de negocio, ideal para decidir el go/no-go.

💡 Ritual sugerido: bug bash de 30 minutos

Cada persona (QA, dev, producto, diseño) toma un charter distinto, todos exploran a la vez y el equipo junta los informes al final. Rápido, barato y revelador antes del lanzamiento.

EMPEZAR

Instalación en 1 minuto

🧩 Extensión de Chrome (recomendado)

Instálala desde la Chrome Web Store, abre el panel lateral con el icono de la brújula y empieza una sesión. Chrome pide acceso a los sitios en el primer uso.

🖥️ Navegador de pruebas (sin tocar tu Chrome)

Con Node.js 18+:

```
npx github:azevedothamirest/qa-scout https://tu-sistema
```

Paso a paso completo, código fuente y tutorial: **github.com/azevedothamirest/qa-scout** (ver `docs/tutorial.md`).

CON IA

Claude explora contigo (MCP)

QA Scout tiene un servidor **MCP** que permite que Claude conduzca la sesión en el navegador de pruebas: lee la pantalla, hace clic, rellena, consulta los errores capturados, anota lo que encuentra y devuelve el informe. La sesión queda en el historial para que la revises.

```
claude mcp add qa-scout -- npx -y github:azevedothamirest/qa-scout mcp
```

"Usa QA Scout para una sesión de 20 minutos en https://staging.mitienda.com: explorar el checkout, con cupones y pago con tarjeta, para descubrir cobros incorrectos. Muéstrame el informe."

1. Lee

Entiende la pantalla por el árbol de accesibilidad.

2. Explora

Hace clic y rellena como una persona; la captura lo graba todo.

3. Anota

Registra bugs, riesgos y dudas y termina con el informe.

Se ejecuta en el terminal (Claude Code o Claude Desktop). El texto de las páginas visitadas se envía a Claude: usa solo entornos y datos de prueba.

¿SE PUEDE USAR EN EL TRABAJO?

Privacidad y seguridad

- ✓ Sin servidor, sin cuenta, sin telemetría: **ninguna solicitud de red** (bloqueada por CSP `connect-src 'none'`).
- ✓ Datos solo en el almacenamiento local del navegador; salen solo cuando copias o descargas algo.
- ✓ Los valores escritos nunca se leen; tokens y contraseñas en las URL se enmascaran.
- ✓ Código abierto (licencia MIT): se puede auditar todo lo que hace.
- ✓ Material para el equipo de seguridad: `docs/seguranca.md`, términos de uso y `SECURITY.md`.

Resumen

QA

Evidencia y trazabilidad de la prueba exploratoria.

Dev

Bugs reproducibles, con contexto técnico.

Producto

Una visión real de la experiencia y los riesgos.

Explora con una misión. Vuelve con evidencia.

github.com/azevedothamirest/qa-scout · Thamires Azevedo · @qa.thami